Order Management (Orders Tab)

> **Audit update — July 16, 2026:** this spec was audited against the current Team Stores Workspace prototype and updated to match its behavior (two sub-tabs, All Orders toolbar, roster-based pending containers, review-modal fee handling, freight-routing gate, and per-player tracking relocated to the My Team(s) tab).
>
> **Terminology update — September 11, 2026:** the Roster tab has been renamed the **My Team(s)** tab. All references to the tab in this spec now use the new name; references to rosters as a data concept (roster groups, standalone rosters, roster architecture) are unchanged.
>
> **Amendment — October 13, 2026:** the order line **snapshots personalization at purchase** and is never re-read from the roster. A new **Governing rule** states it, and **Epic E** specifies the one narrow path by which a pending order line may be updated to match a changed roster. This exists because roster edits are deliberately unrestricted — see *Roster Management (My Team(s) Tab)*, Epics 2 and 4 — and the snapshot is what makes that safe.

Executive Summary & Problem Statement

The Problem

A rep running a team store has orders coming from three directions — their own rep-placed orders, individual player orders for team-required/supplied gear, and fan orders — and no single place to see them. Two operational realities make this acute: most manufacturers enforce **minimum order quantities (MOQs)**, so submitting too early incurs large fees; and reps must know **which rostered players are still missing required gear** before deadlines.

The Solution

An **Orders** tab inside the store manager that consolidates order activity into **two sub-tabs — All Orders and Pending Orders**. Pending Orders is built around the MOQ workflow (let orders pile up, then submit in batches with the minimum-quantity fee surfaced); the per-player ordered-vs-missing view lives in the **My Team(s) tab** as a player drawer. Any order expands into a side-drawer order detail.

Governing rule — the order line is a snapshot

An order line records what was bought. It is a manufacturing instruction somebody has paid for, not a view onto the roster.

- **Personalization is copied onto the order line at purchase.** Jersey number, name on jersey, size, and — on Direct-to-Home stores — the shipping address are written onto the line when the order is placed, and are **never re-read from the player record afterward**.

- **No roster mutation reaches a placed order.** Editing a player's number, renaming them, moving them to another team, deleting them, or clearing the roster entirely leaves every existing order line exactly as it was. There is no cascade, no sync, and no recalculation.
- **This is what lets the roster stay permissive.** *Roster Management* deliberately blocks nothing: reps have said they would rather live with occasional bad data than be locked out of simple corrections. That trade is only safe because the roster cannot reach into production. Without the snapshot, every unrestricted roster edit becomes a silent change to a garment someone has already bought.
- **One narrow exception, in one direction, by explicit action.** A rep may push the current roster values onto a **pending** order line — see Epic E. It is per line, never automatic, and unavailable once the order is submitted.
- **Order → roster write-back is a different thing and is unaffected.** Checkout writing a shipping address onto a player record (*Roster Management*, Epic 3) and a team-gear order creating a Roster Bank capture both run in the opposite direction. Neither is touched by this rule.

User persona & use cases

Who is this for? The store manager (rep) viewing and managing all orders in their team store — their own orders, player orders, and fan orders.

Scenarios encountered Letting pending orders accumulate to hit a factory MOQ before submitting; submitting a shipping group / selected / all pending orders; submitting below MOQ with the minimum-quantity fee itemized in the review modal; entering PO payment + shipping and reviewing items before final submit; opening a player's drawer in the My Team(s) tab to spot missing required gear and send a reminder; **correcting a jersey number on the roster and pushing that correction onto the order before it goes to the factory; discovering after submission that a player's ordered number no longer matches the roster, and warning the parent rather than changing anything;** opening any order to inspect its full detail.

Functional Requirements & Core Workflows

User Stories

- As a rep, I want every order in my store — mine, players', and fans' — manageable from one Orders tab, so that I can manage fulfillment from a single place.
- As a rep, I want to let pending orders accumulate and submit them in batches, so that I hit factory MOQs and avoid below-minimum fees.
- As a rep, I want each shipping group's progress to MOQ and the exact fee for submitting short surfaced up front and itemized at review, so that I never incur a fee by accident.
- As a rep, I want a per-player view of ordered vs. missing required gear in the My Team(s) tab, so that I can quickly chase down players who haven't ordered.

- **As a rep, I want an order to keep the details it was placed with, so that tidying up a roster never changes what the factory makes.**
- **As a rep who corrects a number before anything is submitted, I want to push that correction onto the order, so that the right jersey gets made.**
- As a rep, I want to expand any order into a detail drawer, so that I can inspect items, status, pricing, and shipping.

Epic A — Orders tab & sub-tabs

The Orders page presents **two sub-tabs**: **All Orders** (default) and **Pending Orders**, which carries a live count badge. There is no Player Orders sub-tab — per-player order tracking lives in the My Team(s) tab (Epic C). A search bar (by name, email, or order number) filters within the active sub-tab.

- **All Orders (default):** Given the Orders tab, when it opens, then **All Orders** is shown by default, consolidating the rep's own orders and fan orders into one view, with each row badged by origin (**My Order** / **Fan**). Pending orders appear only in the Pending sub-tab.
- **All Orders toolbar & table:** the view carries search, sort, filter, a date-range picker, a **column-visibility** control, row selection with a bulk bar (**Export Selected**), pagination, and a **CSV Export**. An **Order Status Guide** explains each lifecycle stage, and the factory-order-number link opens the detail drawer directly in Order View.
- **Pending Orders:** Given the Pending sub-tab, then it lists orders that have been **placed in the storefront but not yet approved by the rep and sent to the factory**.

Epic B — Pending Orders & MOQ submission High-impact

Pending orders are grouped into containers that **mirror the store's roster architecture**: League stores render one container per roster group plus one per standalone roster; Single-Team stores get a single container. Each container header shows the team code and rosters, its **shipping destination** (league / group / team managers), and — when below minimum — a **minimum-quantity progress ring** with the % to MOQ and an info tooltip stating the fee that applies if submitted short. This workflow matters because reps deliberately let orders pile up until they hit MOQ to avoid large below-minimum fees.

- **Selection & submit scope:** Given the pending list, the rep can **Submit Group** from any container header, check any number of rows and **Submit Selected (n)**, or **Submit All** from the sticky footer bar.
- **Below-MOQ handling:** a below-MOQ submission proceeds into the same review modal, with the **Minimum Quantity Fee itemized per shipping group and in the order summary** — the rep sees exactly what the shortfall costs before confirming. There is no separate destructive-confirmation step.
- **Review & submit modal:** Given any submission, then a modal opens where the rep reviews **shipping details**, enters payment as a **PO Number**, confirms billing, and reviews the **order**

summary (subtotal, any minimum-quantity fee, total) and the **items grouped by shipping group** (thumbnail, size, quantity, price) before clicking **Confirm · $total**.

- **Freight-routing gate:** Given a submission that includes a group whose fulfillment routing is unresolved (missing group/team manager), then an **Assign Managers** modal blocks the flow — collecting the group manager, or a team manager for every unassigned roster, with address verification — and on completion automatically continues to the review modal.
- **Submission closes the pending window.** The moment an order is submitted it becomes a factory instruction: its personalization snapshot is final and the Epic E update is withdrawn. Nothing in the review modal re-reads the roster, so a roster edit made while the modal is open does not alter what is being submitted.

Epic C — Player order tracking (My Team(s) tab) High-importance

Per-player order tracking lives in the **My Team(s) tab**, not an Orders sub-tab: each rostered player's drawer aggregates all of their **team-required / team-supplied** gear.

- Given a player's drawer, then each required/supplied item shows an **ordered** state (size, quantity, order number, order date, status badge, and price — **$0.00** for team-supplied gear) or a red **"Not ordered"** flag, so the rep can immediately see which players are **missing required gear**.
- Given a player missing required items, then a **Send Reminder** action appears pinned to the drawer footer (only while gear is still un-ordered).
- **Divergence is reported in this drawer.** Given an ordered item whose snapshotted personalization no longer matches the player's current roster values, then the item states both — *ordered as #12 · roster says #7* — and, while the order is still pending, carries the Epic E update action. The display rules are owned by *Roster Management*, Epic 2; the underlying comparison and the update are owned here.
- **Divergence does not change completion.** The item was ordered and remains ordered, so it still counts toward the player's team-gear completion and does not re-arm Send Reminder.

Epic D — Order detail drawer

- Given any order in any sub-tab, when the rep clicks it, then a **side drawer** slides in with the full order detail — product art, status with a production-stage stepper (Submit Date → Order Confirmed → production stages → Delivery), quantity / sport / PO / tracking, delivery, rep/dealership, and item pricing — toggleable between **Order View** and **Item View**.
- **View behavior:** the drawer header carries the Web Order # and Factory Order #, organization, and cart name; it opens in **Item View** by default (or **Order View** when opened from a factory-order link). Order View shows the whole web order — summary cells plus grouped line items — and clicking a line item jumps to that item's Item View.

- **Item View shows the snapshot, and says so where it has drifted.** Personalization displayed in the drawer is the order line's own recorded values, never the player's current record. Where the two differ, Item View notes the current roster value alongside the ordered one, so a rep inspecting an order is never left guessing which number the garment carries.

Epic E — Updating a pending order to match the roster

The one sanctioned path from roster to order. It exists because the pending window is real: orders sit un-submitted while the rep batches toward MOQ (Epic B), so a correction caught in that window can still reach production.

- **Availability.** Given an order line whose snapshotted personalization differs from the player's current roster values, and whose order is still in the **pending** state, then an update action is offered on the divergence line in the player's gear drawer (Epic C) and in the order detail drawer (Epic D).
- **What it does.** Given the rep takes the action, then that **single order line's** snapshot is rewritten from the player's current roster values, the divergence resolves, and the line is thereafter indistinguishable from one placed with those values.
- **Per line, never bulk.** There is no "sync roster to orders" control, no multi-select variant, and no store-wide sweep. A single action must never be capable of rewriting many paid orders at once — that is the outcome the Governing rule exists to prevent, and a bulk control would reintroduce it with one click.
- **Withdrawn on submission.** Given a submitted order, then the action is not offered and the divergence renders as information only. This must be **enforced server-side**, not merely hidden: a stale client must not be able to rewrite a line that has already gone to the factory.
- **It is never automatic and never prompted.** Nothing applies the update on the rep's behalf, and no modal, banner, or attention card asks them to. The system cannot know which value is correct — a rep may have changed the roster number precisely because the player's situation changed — so the choice stays with the rep.
- **Pricing is unaffected.** Personalization carries no price, so an update never alters the line total, the container's MOQ progress, or any figure in the review modal.
- **OPEN — to be confirmed:** whether an update is recorded against the order for support purposes. A line that changed between placement and submission is the kind of thing a parent dispute turns on, and the order currently has no amendment history. Recommend a minimal record — what changed, when, and by whom — resolved with whoever owns order history.

Metrics & Success Criteria

- Share of submissions made at/above MOQ (below-MOQ fee avoidance)
- Minimum-quantity fees incurred (itemized at review)

- Time from storefront order → rep submission to factory
- Required-gear completion rate per roster before deadline (tracked via My Team(s)-tab player drawers)
- Reminder → player-order conversion
- **Order lines carrying a divergence at the point of submission** — gear that will arrive not matching the roster. The number that matters, because each one is a potential replacement
- **Pending-order updates taken, as a share of divergences arising while still pending** — whether the window is being used; a low share means the divergence line is not visible enough
- **Divergences first arising after submission** — cases the pending window could never have caught, and the argument for surfacing roster edits earlier
- **Replacement or re-order volume attributable to mismatched personalization** — the real-world cost the snapshot rule is trading against, and the test of whether permissive roster editing is affordable

---

Dependencies & Technical Constraints

Dependency & Technical Constraint

Requires an **order service** spanning rep-placed, player, and fan orders with a **storefront → pending → submitted** lifecycle. Pending submission needs MOQ thresholds and **minimum-quantity fee** calculation per shipping container keyed to the store's **roster architecture** (roster groups / standalone rosters / single team), **batch submission** (group / selected / all), and a review modal capturing **PO-number payment** + shipping and computing subtotal / fee / total with the fee itemized per group. Submission is gated on **resolved freight routing** — an assign-managers flow with address verification must be able to interrupt and resume the submit. Player order tracking requires a **roster join** surfaced in the My Team(s) tab's player drawers (ordered-vs-missing required/supplied gear) plus a reminder channel. The drawer requires **order-detail + production-stage** data. All Orders additionally needs **CSV export** (full and selected) and per-user column-visibility preferences. Shared internal order / roster / factory services.

The order line must store personalization as its own fields, not as a reference to the player record. Jersey number, name on jersey, size, and the Direct-to-Home shipping address are written at purchase and owned by the line thereafter. No read path may resolve them through the roster — not the order detail drawer, not the review modal, not the PO / factory export, and not any report. A foreign key to the player record may be retained for the roster join that drives per-player tracking, but it must not be the source of the printed values. This is the single constraint the permissive roster model in *Roster Management* depends on.

The **divergence read** is a derived comparison between a player's current roster values and the snapshot on each of their order lines. It must be computed rather than stored as a flag, so that it resolves itself when either side changes — including when the rep edits the roster back to the original value. It is consumed by the player gear drawer (Epic C), the order detail drawer (Epic D), and nothing else; it is deliberately not surfaced on Home, because the condition disappears at submission, which is precisely when the rep loses the ability to act on it.

The **pending-order update** (Epic E) rewrites one order line's personalization snapshot from the current roster values. It must be **scoped to a single line**, with no bulk endpoint existing at all — the absence of one is a requirement, not an omission — and must be **refused server-side** for any order no longer in the pending state. Whether the amendment is recorded in an order history is an open dependency (Epic E) and should be settled before build, since no amendment history exists today.